

使用 Google Cloud Lakehouse 和 Knowledge Catalog 建構受控 Iceberg 湖倉

來源：<https://codelabs.developers.google.com/governed-lakehouse-compute-delegation?hl=zh-tw>

步驟數：8

在本程式碼研究室中，您將使用 Apache Iceberg、BigQuery 和 Knowledge Catalog 建構受控資料湖倉。您將模擬實際的企業情境，解決重要的資料挑戰：在 Knowledge Catalog 中定義動態資料遮蓋政策一次，並證明這些規則如何在完全不同的執行引擎中普遍強制執行。

目錄

1. [簡介](#)
2. [設定和需求](#)
3. [透過 Lakehouse 建立原生 Iceberg 資料表](#)
4. [集中式控管：使用 Python 定義政策](#)
5. [驗證 Knowledge Catalog 政策](#)
6. [自動化資料歷程](#)
7. [清除所用資源](#)
8. [恭喜！](#)

1. 簡介

在現代企業資料雲端中，資料會儲存在各種實體儲存系統，因此安全防護分散各處，造成巨大的架構挑戰。

當資料以 Parquet 等開放原始碼格式實際儲存在 [Google Cloud Storage](#) 中，並由 BigQuery SQL 或 Apache Spark 等多種不同引擎查詢時，如何確保財務交易金額等機密資料受到一致的保護？

在本程式碼研究室中，您將建構受管理資料湖倉架構，並使用 [Apache Iceberg 資料表](#)、[BigQuery](#) 和 [Knowledge Catalog](#) 解決這些問題。您將使用基礎架構即程式碼 (IaC) 定義零信任安全政策，以及如何在不同運算引擎中動態強制執行這些政策。

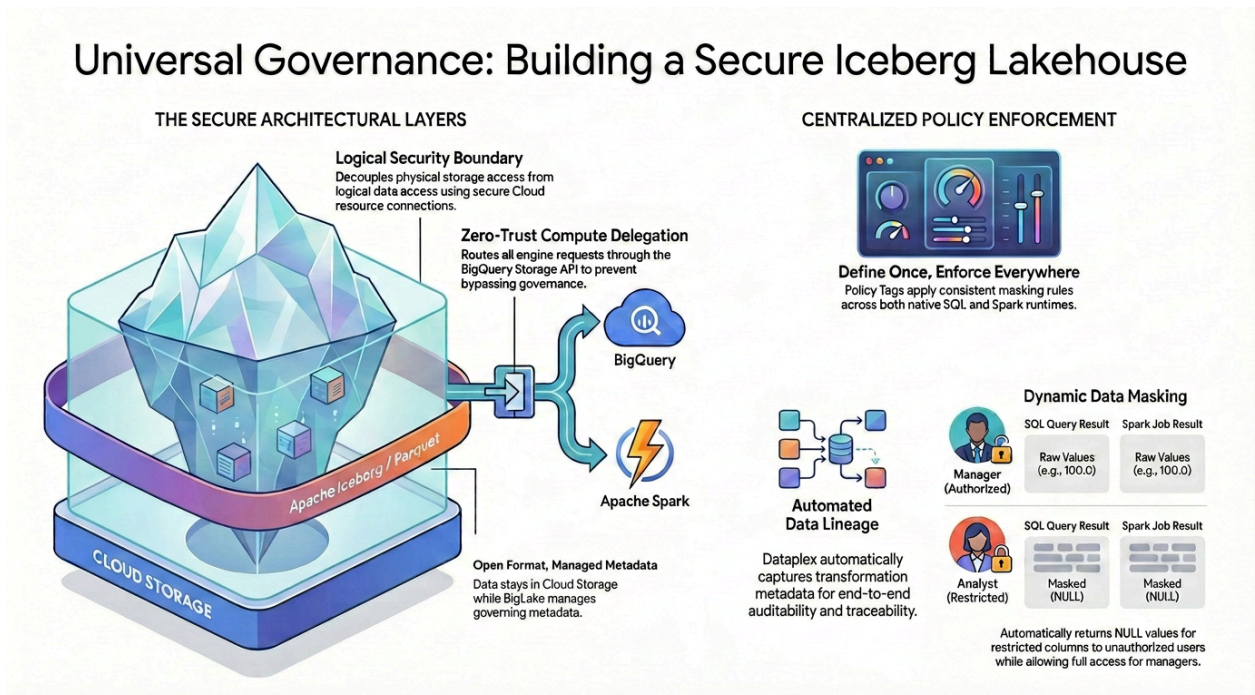
必要條件

- 已啟用計費功能的 Google Cloud 專案。
- 瞭解 SQL、IAM 和 Cloud Storage 的基本概念。

課程內容

- 如何在 [BigQuery](#) 中為 [Apache Iceberg](#) 建立 [Google Cloud Lakehouse](#) 資料表，而 Cloud Storage 本身會保存資料。
- 如何使用 [政策標記](#)，強制執行集中式資料政策，以確保資料欄層級安全和資料遮蓋。
- 如何使用 [Cloud 資源連線](#)，將實體儲存空間存取權與邏輯資料存取權分離。
- 如何使用 [Google Cloud Serverless for Apache Spark](#) 強制執行零信任運算委派，確保開放原始碼引擎無法規避控管。
- 如何以圖表呈現自動化 [資料歷程](#)。

架構總覽：Iceberg 的通用控管



若要對開放原始碼資料格式進行精細的存取控管 (例如資料欄層級安全防護和資料遮蓋)，您必須建立嚴格且統一的安全架構。

如圖所示，這個受控的 Lakehouse 模式有兩大支柱，可解決安全防護分散的問題：

安全架構層 (左側)

您不必允許使用者或外部引擎直接存取 Cloud Storage (僅支援廣泛的值區層級安全性)，而是建立安全基礎。

- **開放格式、代管中繼資料：**資料會以開放的 Apache Iceberg (Parquet) 格式實際儲存在 Cloud Storage 中，而 Lakehouse 會順暢地管理控管中繼資料。
- **邏輯安全界線：**您可以使用安全的 Cloud 資源連結，將實體儲存空間存取權與邏輯資料存取權分離。系統絕不會授予使用者直接存取原始 GCS 檔案的 IAM 實體存取權。
- **零信任運算委派：**為確保執行引擎不會規避管理規則，所有資料讀取要求都會嚴格透過 BigQuery Storage API 傳送。無論查詢是來自原生 BigQuery SQL 或開放原始碼 Apache Spark，都適用這項限制。

集中式政策強制執行 (右)

有了安全基礎，Knowledge Catalog 就能做為統一的治理大腦：

- **定義一次，隨處強制執行：**您只需要在 Knowledge Catalog 中定義一次政策標記，架構就會在所有支援的執行階段中，普遍套用一致的遮蓋規則。
 - **動態資料遮蓋：**系統會在查詢資料時，即時評估使用者身分。授權使用者在 SQL 和 Spark 中都會看到原始的未遮蓋值 (例如 100.0)，但受限使用者在兩個引擎中，都會自動收到受限資料欄的遮蓋 NULL 值。
 - **自動化資料沿襲：**隨著資料流動和轉換，Knowledge Catalog 會自動擷取轉換中繼資料，提供內建的端對端稽核和追蹤功能，不需自訂記錄程式碼。
-

步驟 2 · 約 0 分鐘

2. 設定和需求

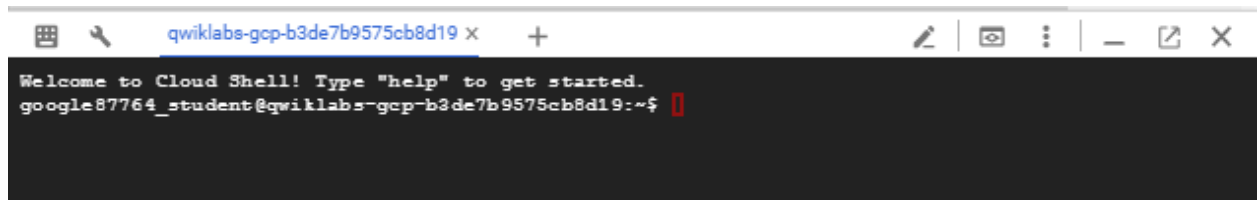
啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

在 [Google Cloud 控制台](#) 中，點選右上工具列的 Cloud Shell 圖示：



佈建並連線至環境的作業需要一些時間才能完成。完成後，您應該會看到如下的內容：



這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。您可以在瀏覽器中完成本程式碼研究室的所有作業。您不需要安裝任何軟體。

初始化環境

開啟 Cloud Shell 並設定專案變數，確保所有指令都以正確的基礎架構為目標。

```
export PROJECT_ID=$(gcloud config get-value project)
export REGION="us-central1"
export ICEBERG_BUCKET="iceberg-retail-demo-${PROJECT_ID}"
export DATASET_ID="lakehouse_retail_demo"
export CONN_NAME="iceberg-bq-conn-demo"
```

接著定義兩個目標對象。

```
export USER_ANALYST="retail-analyst-demo"
export EMAIL_ANALYST="${USER_ANALYST}@${PROJECT_ID}.iam.gserviceaccount.com"

export USER_MANAGER="retail-manager-demo"
export EMAIL_MANAGER="${USER_MANAGER}@${PROJECT_ID}.iam.gserviceaccount.com"
export CURRENT_USER=$(gcloud config get-value account)
```

啟用 API

啟用必要的 Google Cloud 服務。

```
gcloud services enable \  
  bigquery.googleapis.com \  
  bigqueryconnection.googleapis.com \  
  datacatalog.googleapis.com \  
  bigquerydatapolicy.googleapis.com \  
  datalineage.googleapis.com \  
  dataplex.googleapis.com \  
  dataproc.googleapis.com \  
  storage-component.googleapis.com
```

下載程式碼研究室原始碼

為避免 Cloud Shell 雜亂無章，您將執行**稀疏簽出**，只從 Google Cloud DevRel 存放區下載本程式碼研究室所需的 Python 指令碼。

```
# Shallow clone without full history  
git clone --depth 1 --filter=blob:none --sparse  
https://github.com/GoogleCloudPlatform/devrel-demos.git  
cd devrel-demos  
  
# Download only the specific folder  
git sparse-checkout set data-analytics/governed-lakehouse  
cd data-analytics/governed-lakehouse
```

建立儲存空間

建立 bucket，存放受到嚴格控管的高度安全 Iceberg 資料。

```
gcloud storage buckets create gs://${ICEBERG_BUCKET} --location=${REGION}
```

準備身分識別與安全性

設定 Cloud 資源連結。只有這個實體擁有永久實體 IAM 金鑰，可讀取原始 Iceberg 檔案。

```
# Create the BigQuery connection
bq mk --connection \
  --connection_type=CLOUD_RESOURCE \
  --location=${REGION} \
  ${CONN_NAME}

# Retrieve the connection's automatically generated Service Account
export BQ_CONN_SVC_ACCT=$(bq show --format=json --connection ${REGION}.${CONN_NAME} \
  | jq -r '.cloudResource.serviceAccountId')

# Grant Storage Object Admin to the connection for the Iceberg bucket
gcloud storage buckets add-iam-policy-binding gs://${ICEBERG_BUCKET} \
  --member="serviceAccount:${BQ_CONN_SVC_ACCT}" \
  --role="roles/storage.objectAdmin" \
  --quiet
```

接著設定使用者輪廓。使用者獲得的是邏輯存取權，而非實體儲存空間存取權。為避免 IAM 傳播延遲導致錯誤，請先建立帳戶，等待幾秒鐘，然後再指派角色。

```
echo "Creating Service Accounts..."
for USER in "${USER_ANALYST}" "${USER_MANAGER}"; do
    gcloud iam service-accounts create ${USER} --display-name="Lakehouse ${USER}"
done

echo "⌚ Waiting 15 seconds for IAM propagation..."
sleep 15

echo "Granting IAM Roles to Service Accounts..."
for USER in "${USER_ANALYST}" "${USER_MANAGER}"; do
    EMAIL="${USER}@${PROJECT_ID}.iam.gserviceaccount.com"

    # Allow Cloud Shell to impersonate them for testing
    gcloud iam service-accounts add-iam-policy-binding ${EMAIL} \
        --member="user:${CURRENT_USER}" \
        --role="roles/iam.serviceAccountTokenCreator" \
        --quiet

    # Allow logical viewing of the catalog, querying, and running Dataproc jobs
    for ROLE in "roles/datacatalog.viewer" "roles/bigquery.dataViewer" "roles/bigquery.user"
    "roles/bigquery.connectionUser" "roles/serviceusage.serviceUsageConsumer"
    "roles/dataproc.worker"; do
        gcloud projects add-iam-policy-binding ${PROJECT_ID} \
            --member="serviceAccount:${EMAIL}" \
            --role="${ROLE}" \
            --quiet
    done
done

# Grant the Manager data creation rights
gcloud projects add-iam-policy-binding ${PROJECT_ID} \
    --member="serviceAccount:${EMAIL_MANAGER}" \
    --role="roles/bigquery.dataEditor" \
    --quiet

echo "✅ Identity and Security setup completed!"
```

步驟 3 · 約 0 分鐘

3. 透過 Lakehouse 建立原生 Iceberg 資料表

您將使用 Lakehouse 的原生功能建立代管 Iceberg 資料表。

建立 BigQuery 資料集

首先，請建立 BigQuery 資料集，以邏輯方式將 Iceberg 資料表分組。

```
echo "Creating BigQuery Dataset..."
bq mk --location=${REGION} --dataset ${PROJECT_ID}:${DATASET_ID}
```

建立 Iceberg 資料表

接著，請執行下列指令來建立資料表。請注意 `OPTIONS` 區塊，我們在其中指定 `table_format = 'ICEBERG'`，並直接將其對應至 Cloud Storage 值區和連線。

```
echo "Creating Iceberg tables..."

# Inventory table
bq query --use_legacy_sql=false \
"CREATE OR REPLACE TABLE \`${PROJECT_ID}.${DATASET_ID}.inventory`\` (
  product_id INT64,
  product_name STRING,
  stock_count INT64
)
WITH CONNECTION \`${REGION}.${CONN_NAME}`\`
OPTIONS (
  file_format = 'PARQUET',
  table_format = 'ICEBERG',
  storage_uri = 'gs://${ICEBERG_BUCKET}/inventory/'
);"

# Transactions table
bq query --use_legacy_sql=false \
"CREATE OR REPLACE TABLE \`${PROJECT_ID}.${DATASET_ID}.transactions`\` (
  id INT64,
  item STRING,
  amount FLOAT64,
  transaction_date DATE
)
WITH CONNECTION \`${REGION}.${CONN_NAME}`\`
OPTIONS (
  file_format = 'PARQUET',
  table_format = 'ICEBERG',
  storage_uri = 'gs://${ICEBERG_BUCKET}/transactions/'
);"
```

在資料表中填入資料

最後，將範例資料插入新建立的 Iceberg 資料表。

```
echo "Inserting data into Iceberg tables..."

# Insert into Inventory table
bq query --use_legacy_sql=false \
"INSERT INTO \`${PROJECT_ID}.${DATASET_ID}.inventory\` (product_id, product_name,
stock_count)
VALUES (101, 'Widget A', 500), (102, 'Widget B', 250), (103, 'Widget C', 800);"

# Insert into Transactions table
bq query --use_legacy_sql=false \
"INSERT INTO \`${PROJECT_ID}.${DATASET_ID}.transactions\` (id, item, amount,
transaction_date)
VALUES
  (1, 'Widget A', 100.0, DATE '2024-01-01'),
  (2, 'Widget B', 150.0, DATE '2024-01-02'),
  (3, 'Widget C', 50.0, DATE '2024-01-03');"
```

現在您有兩個功能齊全的 Iceberg 資料表。資料湖倉會管理中繼資料，但實體 Parquet 檔案會安全地儲存在 GCS 值區中！

模擬 ETL 管道

在實際情境中，原始資料通常會匯總到摘要表格中，用於業務報表。我們現在扮演資料工程師，根據原始交易資料建立每日銷售摘要表。

(注意：請立即執行這個步驟，讓 *Google Cloud* 有足夠時間處理背景中繼資料。您會在程式碼研究室的後續章節中瞭解這項操作的重要性！)

```
echo "Creating transactions summary table..."
bq query --use_legacy_sql=false \
"CREATE TABLE \`${PROJECT_ID}.${DATASET_ID}.transactions_summary\` AS
SELECT transaction_date, SUM(amount) as total_sales, COUNT(id) as transaction_count
FROM \`${PROJECT_ID}.${DATASET_ID}.transactions\`
GROUP BY transaction_date;"
```

步驟 4 · 約 0 分鐘

4. 集中式控管：使用 Python 定義政策

在正式環境中，透過使用者介面設定管理政策很難擴充及維護。強烈建議改用基礎架構即程式碼 (IaC)。

在本節中，您將使用 Google Cloud Python SDK，以程式輔助的方式逐步建立及強制執行零信任控管規則。

設定 Python 環境

首先，請設定獨立的 Python 環境 (`venv`)，避免程式庫衝突，並安裝必要的 Google Cloud SDK。

在 Cloud Shell 中執行下列指令：

```
# Create and activate a virtual environment
python3 -m venv lakehouse_env
source lakehouse_env/bin/activate

# Install required Knowledge Catalog and BigQuery governance libraries
pip install google-cloud-datacatalog google-cloud-bigquery-datapolicies google-cloud-bigquery
--quiet

echo "✅ Python environment is ready!"
```

建立分類和政策標記

分類是邏輯容器，政策標記則是您要附加至機密資料欄的特定標籤。如要強制執行資料欄層級安全防護機制，您必須先建立邏輯容器 (分類) 和特定標籤 (政策標記)。

查看 `1_create_taxonomy.py` 內部，您會看到下列核心邏輯：

```
# Create Taxonomy with Fine-Grained Access Control enabled
taxonomy = datacatalog_v1.Taxonomy(
    display_name="BusinessCritical",
    activated_policy_types=[datacatalog_v1.Taxonomy.PolicyType.FINE_GRAINED_ACCESS_CONTROL]
)
created_taxonomy = client.create_taxonomy(parent=parent, taxonomy=taxonomy)

# Create Policy Tag inside the Taxonomy
policy_tag = datacatalog_v1.PolicyTag(display_name="RestrictedFinancial")
created_policy_tag = client.create_policy_tag(parent=created_taxonomy.name,
policy_tag=policy_tag)
```

明確設定 `FINE_GRAINED_ACCESS_CONTROL` 政策類型後，標準中繼資料標記就會轉換為嚴格的零信任安全邊界。預設情況下，所有使用者都無法存取含有這個標記的資料欄。

執行指令碼來建立資源：

```
python 1_create_taxonomy.py
```

設定遮蓋規則 (資料政策)

現在，請定義沒有權限的使用者查詢已加上標記的資料欄時會發生什麼情況。您將建立**資料政策**，強制傳回 `NULL` 值，並將這項規則附加至**分析師角色**。

在 `2_create_masking.py` 中，指令碼會動態查閱您剛建立的政策標記 ID，並套用資料政策：

```
# Define a Masking Policy that always returns NULL
data_policy = bigquery_datapolicies_v1.DataPolicy(
    data_policy_id="mask_financial_null",
    policy_tag=policy_tag_id,
    data_policy_type=bigquery_datapolicies_v1.DataPolicy.DataPolicyType.DATA_MASKING_POLICY,
    data_masking_policy=bigquery_datapolicies_v1.DataMaskingPolicy(

predefined_expression=bigquery_datapolicies_v1.DataMaskingPolicy.PredefinedExpression.ALWAYS_
NULL
    )
)

# ... (Policy creation code) ...

# Bind the Masked Reader role to the Analyst
iam_policy.bindings.add(
    role="roles/bigquerydatapolicy.maskedReader",
    members=[f"serviceAccount:{analyst_email}"]
)
```

這段程式碼會以程式輔助方式建立規則，強制將基礎值傳回為 `NULL`。接著，系統會將 `maskedReader` IAM 角色指派給「分析師」角色，確保他們只會看到資料的遮蓋版本。

執行指令碼來設定遮蓋規則：

```
python 2_create_masking.py
```

授予精細存取權

由於我們採用零信任設定，目前無人可讀取標記的資料欄。您必須明確授權管理員和個人帳戶存取權。

在 `3_grant_access.py` 中，您可以修改政策標記本身的 IAM 政策：

```
# Grant original data read access
iam_policy.bindings.add(
    role="roles/datacatalog.categoryFineGrainedReader",
    members=[f"serviceAccount:{manager_email}", f"user:{current_user}"]
)
client.set_iam_policy(request=iam_policy_pb2.SetIamPolicyRequest(resource=policy_tag_id,
policy=iam_policy))
```

新增 `categoryFineGrainedReader` 角色後，這些特定主體就能略過遮蓋規則，讀取未經遮蓋的原始資料。

執行指令碼來授予存取權：

```
python 3_grant_access.py
```

將政策標記附加至 BigQuery 資料表

最後，您必須將這個邏輯政策代碼附加至實體 Iceberg 資料表結構定義。

請看一下 `4_attach_tag.py`。這個指令碼會擷取 BigQuery 資料表結構定義、逐一檢查欄位，然後將標記附加至 `amount` 欄：

```
new_schema = []
for field in table.schema:
    if field.name == 'amount':
        # Wrap the Policy Tag ID and attach it to the column
        policy_tags_list = bigquery.PolicyTagList(names=[policy_tag_id])
        new_field = bigquery.SchemaField(
            name=field.name, field_type=field.field_type, mode=field.mode,
            description=field.description, policy_tags=policy_tags_list
        )
        new_schema.append(new_field)
    else:
        new_schema.append(field)

# Update the table schema in BigQuery
table.schema = new_schema
client.update_table(table, ["schema"])
```

套用這個結構定義更新後，Lakehouse 會立即將 Knowledge Catalog 邏輯標記橋接至儲存在 Cloud Storage bucket 中的實體 Parquet 檔案。

執行指令碼來更新資料表結構定義：

```
python 4_attach_tag.py
```

⚠ 警告：

IAM 和政策標記需要 2 到 5 分鐘，才能在全球的 Google Cloud 中傳播。請勿立即進行下一個步驟。請等待 3 分鐘，確保不會發生錯誤的「Access Denied」錯誤。

5. 驗證 Knowledge Catalog 政策

現在要測試集中式控管是否正常運作。您將在兩個不同的引擎中測試這項功能，證明 Knowledge Catalog 政策普遍適用。

使用 BigQuery 原生 SQL 驗證

首先，您會使用 Cloud Shell 擔任這兩個角色，並使用 BigQuery 的原生 SQL 引擎查詢資料表。

以管理員 (具備特殊權限的使用者) 身分進行測試：

```
# Impersonate the manager
gcloud config set auth/impersonate_service_account ${EMAIL_MANAGER}

# Query the transactions table
bq query --use_legacy_sql=false "SELECT * FROM \`${PROJECT_ID}.${DATASET_ID}.transactions\`"
```

由於管理員具備精細讀取者角色，因此會顯示原始金額值

id	item	amount	transaction_date
1	Widget A	100.0	2024-01-01
3	Widget C	50.0	2024-01-03
2	Widget B	150.0	2024-01-02

以分析師 (受限制的使用者) 身分測試：

```
gcloud config set auth/impersonate_service_account ${EMAIL_ANALYST}

bq query --use_legacy_sql=false "SELECT * FROM \`${PROJECT_ID}.${DATASET_ID}.transactions\`"
```

由於 Knowledge Catalog 遮蓋規則的關係，金額欄會為每個資料列傳回 `NULL`。

id	item	amount	transaction_date
1	Widget A	NULL	2024-01-01
3	Widget C	NULL	2024-01-03
2	Widget B	NULL	2024-01-02

還原身分

清理 Cloud Shell 驗證狀態，返回管理員使用者。

```
# Unset impersonation
gcloud config unset auth/impersonate_service_account
```

使用 Apache Spark (運算委派) 進行驗證

如果資料科學家使用 Apache Spark 讀取這個資料表，會發生什麼情況？如果 Spark 直接讀取實體 GCS Parquet 檔案，由於 Cloud Storage 只瞭解 bucket 層級的權限，因此會完全略過 Knowledge Catalog 遮蓋規則。

為避免發生這種情況，請使用 [Spark-BigQuery 連接器](#) 強制執行運算委派。這個連接器可做為安全橋樑，透過 BigQuery Storage API 轉送 Spark 讀取要求，以便在將任何資料傳送至 Spark 叢集之前，動態評估 Knowledge Catalog 管理規則。

查看您下載的 `read_transactions.py` 指令碼中的核心邏輯：

```
# Reading data via Compute Delegation (Knowledge Catalog policies are applied dynamically here)
df = spark.read \
    .format("bigquery") \
    .option("table", f"{project_id}.{dataset_id}.{table_name}") \
    .load()

print("\n=== 📊 Data Preview ===")
df.show(truncate=False)
```

請注意，我們並未將 Spark 指向 Iceberg 檔案的 `gs://` 路徑。指定 `.format("bigquery")` 後，BigQuery Storage API 會攔截讀取要求、檢查執行 Spark 工作的使用者身分、套用 Knowledge Catalog 遮蓋規則，並只將授權資料傳回 Spark DataFrame。

將這個 PySpark 指令碼上傳至 Cloud Storage bucket，讓 Dataproc 可以存取：

```
# Upload script to GCS
gsutil cp read_transactions.py gs://${ICEBERG_BUCKET}/scripts/read_transactions.py
```

以管理員身分執行 Spark：

您將使用 Google Cloud Serverless for Apache Spark。這項代管服務可讓您直接執行 Spark 工作負載，不必佈建、設定或管理專屬叢集。

```
echo "🚀 Submitting Dataproc Serverless Job as [MANAGER]..."
gcloud dataproc batches submit pyspark gs://${ICEBERG_BUCKET}/scripts/read_transactions.py \
    --project=${PROJECT_ID} \
    --region=${REGION} \
    --service-account=${EMAIL_MANAGER} \
    --version=2.3 \
    -- ${PROJECT_ID} ${DATASET_ID} \
    --format="value(name)"
```

查看終端機中的工作輸出記錄。由於管理員具有「精細讀取者」角色，Spark 成功擷取了未經過遮蓋的原始金額。

```
=== 📊 Data Preview ===
+---+-----+-----+-----+
|id |item   |amount|transaction_date |
+---+-----+-----+-----+
|1  |Widget A|100.0 |2024-01-01       |
|2  |Widget B|150.0 |2024-01-02       |
|3  |Widget C|50.0  |2024-01-03       |
+---+-----+-----+-----+
```

以分析師身分執行 Spark：

現在，請提交完全相同的 Spark 工作，但這次要模擬分析師角色。

```
echo "🚀 Submitting Dataproc Serverless Job as [ANALYST]..."
gcloud dataproc batches submit pyspark gs://${ICEBERG_BUCKET}/scripts/read_transactions.py \
  --project=${PROJECT_ID} \
  --region=${REGION} \
  --service-account=${EMAIL_ANALYST} \
  --version=2.3 \
  -- ${PROJECT_ID} ${DATASET_ID} \
  --format="value(name)"
```

再次檢查記錄。即使分析師執行完全相同的 Spark 程式碼，BigQuery Storage API 仍會攔截要求並強制執行 Knowledge Catalog 政策。分析師的 Spark DataFrame 會顯示金額的 `null`！

```
=== 📊 Data Preview ===
+---+-----+-----+-----+
|id |item   |amount|transaction_date |
+---+-----+-----+-----+
|1  |Widget A|null   |2024-01-01       |
|2  |Widget B|null   |2024-01-02       |
|3  |Widget C|null   |2024-01-03       |
+---+-----+-----+-----+
```

架構取舍：BigQuery SQL 與 Spark

您剛才證明，無論使用哪個引擎，結果都相同！已成功強制執行 Knowledge Catalog 政策。但在實際執行中，您應該使用哪一個？

- **BigQuery SQL**：適合需要使用 SQL 引擎，並直接在原地執行運算的流程。非常適合快速分析和商業智慧。
- **Apache Spark**：可使用 Python 處理更複雜的工作負載，因此非常適合用於進階機器學習管道或舊版 Hadoop 程式碼。

重點：無論使用哪個引擎，只要強制執行運算委派，集中式零信任治理層就絕不會遭到規避！

6. 自動化資料歷程

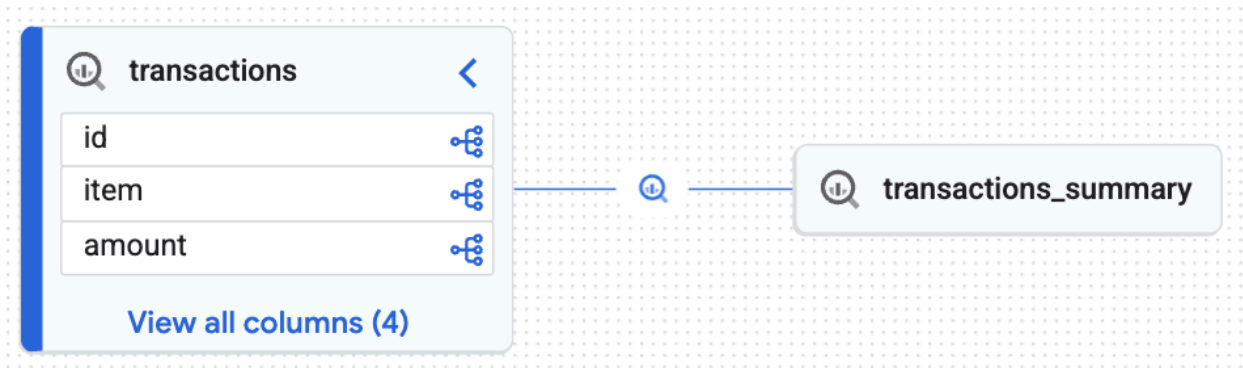
在任何企業資料架構中，瞭解資料的來源和變更方式，對於法規遵循、偵錯及建立信任感至關重要。這個概念稱為「資料沿襲」。這項功能可回答基本問題，例如：「如果管理員查看每日銷售報表，系統會使用哪些原始資料表計算這些數字？」

傳統上，追蹤這個生命週期需要資料工程師手動編寫自訂記錄程式碼，或使用複雜的第三方工具剖析 SQL 指令碼。不過，在受控管的 Google Cloud Lakehouse 中，這項追蹤功能是內建的，完全不需要手動操作。

還記得您在程式碼研究室稍早從原始交易資料表建立的 `transactions_summary` 資料表嗎？BigQuery 執行該 `CREATE TABLE AS SELECT` 陳述式時，運算引擎會自動擷取轉換中繼資料，並傳送至 Knowledge Catalog。我們來看看結果。

以圖表呈現歷程

1. 在 Google Cloud 控制台中，依序前往「Knowledge Catalog」>「Search」。
2. 在搜尋列中輸入 `lakehouse_retail_demo.transactions`，然後按一下資料表。
3. 按一下「歷程」分頁標籤。



您會看到知識引擎生成的互動式圖表，證明目標資料表 (`transactions_summary`) 是衍生自原始受管理 Iceberg 資料表 (`transactions`)。您已完成資料稽核所需的端對端追蹤。

步驟 7 · 約 0 分鐘

7. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本程式碼研究室所用資源的費用，請按照下列步驟操作。

移除 Knowledge Catalog 控管資源

刪除 BigQuery 資料集或 Cloud Storage 值區前，請務必先移除邏輯控管規則。如果您查看存放區中的 `cleanup_governance.py` 指令碼，會看到以下拆解順序：

```
# 1. Delete Data Policy
data_policy_name = f"{parent_loc}/dataPolicies/mask_financial_null"
dp_client.delete_data_policy(name=data_policy_name)

# 2. Find and Delete Taxonomy (This auto-deletes child Policy Tags)
taxonomies = catalog_client.list_taxonomies(parent=parent_loc)
taxonomy_id = next((t.name for t in taxonomies if t.display_name == "BusinessCritical"),
None)
catalog_client.delete_taxonomy(name=taxonomy_id)
```

這裡的順序至關重要。由於資料政策 (遮蓋規則) 依附於政策標記，因此腳本會先刪除資料政策。移除政策後，刪除父項分類時，系統會自動連帶刪除所有基礎政策標記，不會觸發資源依附元件錯誤。

執行 Python 清理指令碼：

```
python cleanup_governance.py
```

移除身分、儲存空間和運算資產

現在治理層已分離，您可以安全地刪除 BigQuery 資料表、Cloud Storage bucket、服務帳戶和本機 Python 環境。

在 Cloud Shell 中複製並執行下列完整的清除區塊：

```

echo "Deleting Service Accounts and Impersonation Bindings..."
export CURRENT_USER=$(gcloud config get-value account)

for USER in "${USER_ANALYST}" "${USER_MANAGER}"; do
    EMAIL="${USER}@${PROJECT_ID}.iam.gserviceaccount.com"

    # Remove impersonation binding
    gcloud iam service-accounts remove-iam-policy-binding ${EMAIL} \
        --member="user:${CURRENT_USER}" \
        --role="roles/iam.serviceAccountTokenCreator" \
        --quiet > /dev/null 2>&1

    # Delete the Service Account
    gcloud iam service-accounts delete ${EMAIL} --quiet
done

echo "Removing BigQuery Dataset and Tables..."
bq rm -f ${DATASET_ID}.transactions_summary
bq rm -f ${DATASET_ID}.transactions
bq rm -f ${DATASET_ID}.inventory
bq rm -f -d ${DATASET_ID}

echo "Removing BigQuery Cloud Resource Connection..."
bq rm --connection --location=${REGION} ${CONN_NAME}

echo "Removing Iceberg Cloud Storage Bucket..."
gcloud storage rm --recursive gs://${ICEBERG_BUCKET} --quiet

echo "Removing Auto-generated Dataproc Staging & Temp Buckets..."
for BUCKET in $(gcloud storage ls | grep -E "gs://dataproc-(staging|temp)-${REGION}"); do
    gcloud storage rm --recursive $BUCKET --quiet
done

echo "Deactivating and removing the local Python environment..."
deactivate
cd ../../
rm -rf devrel-demos

echo "✅ Clean up completed successfully!"

```

完成這些步驟後，即可確保專案中沒有任何孤立資源或隱藏政策。

步驟 8 · 約 0 分鐘

8. 恭喜！

您已成功導入可探索的完全控管式資料湖倉。

您學到下列事項：

- **原生 Iceberg 整合：** Lakehouse 可原生管理開放原始碼 Iceberg 資料表，同時將實體檔案安全地儲存在 Cloud Storage 中。
- **運算委派作業可確保安全：** 透過 BigQuery Storage API 傳送查詢，即可對實體檔案強制執行精細的動態遮蓋，原生檔案無法限制部分存取權。
- **與引擎無關的控管：** 您只需定義一次政策標記規則，無論是透過原生 SQL 或 Apache Spark 執行階段查詢，系統都會普遍強制執行這些規則。
- **資料可探索性：** 知識引擎會自動追蹤資料沿襲，提供必要的企業稽核能力。

後續步驟

- **探索進階存取控管：** 如要實作更複雜的安全情境，請參閱[自訂 Lakehouse 的官方說明文件](#)。
 - **控管生成式 AI 的非結構化資料：** 探索 [Object Table](#)。將這個確切的安全橋接模式擴展至 Cloud Storage 中的非結構化檔案 (PDF、圖片)，為 Vertex AI 和 RAG 管道建立安全且受控管的資料基礎。
-